Module 3: The Transport Layer

Transport services and ports, TCP and UDP, connection management, flow control, congestion control, and quality of service

Exam: Wednesday 18 November 2026

Exam Focus

Scope. Syllabus 3.1 to 3.4: introduction to the transport layer; TCP and UDP (features, differences, use cases); flow control and congestion control mechanisms; and quality of service.

This module carries CO3 and appears in **every** paper as a 10-mark question, most often as "compare TCP and UDP" or "explain congestion control". It is also the module where CAE-II questions concentrate, so treat it as guaranteed.

Contents

1 Transport layer services **2**
- 1.1 Process-to-process delivery . . . 2
- 1.2 Port numbers . . . 2

2 UDP **2**

3 TCP **3**
- 3.1 Features . . . 3
- 3.2 TCP segment header . . . 3
- 3.3 Connection management . . . 4

4 TCP versus UDP **4**
- 4.1 What is the difference between connection-oriented and connectionless services . . . 5

5 Flow control **5**
- 5.1 The sliding window in TCP . . . 5

6 Congestion control **5**
- 6.1 How congestion occurs . . . 6
- 6.2 Slow start, AIMD and fast recovery . . . 6
- 6.3 Modern variants . . . 7

7 Quality of service **7**
- 7.1 Techniques . . . 7

8 Answer skeletons **8**

9 Self-test **8**

10 Last-minute page **9**

Transport layer services

Process-to-process delivery

The network layer delivers a packet to a *host*. The transport layer delivers a segment to a *process* on that host. That difference is the whole point of the layer.

Function	Description
Process-to-process delivery	Deliver to the correct application, identified by a port number.
Multiplexing	Many applications send through one transport entity; each segment carries its source port.
Demultiplexing	The receiver uses the destination port to hand each segment to the right socket.
Segmentation and reassembly	Split the application stream into segments that fit the MTU and reassemble in order.
Connection control	Connectionless (UDP) or connection-oriented (TCP).
Flow control	Protect the receiver from being overrun.
Error control	Detect loss and corruption end to end and retransmit.
Congestion control	Adapt the sending rate to the state of the network.

Port numbers

A port is a 16-bit number, 0 to 65535, that identifies an application endpoint.

Range	Name	Meaning
0–1023	Well-known	Assigned by IANA to standard servers: 20/21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 67/68 DHCP, 80 HTTP, 110 POP3, 143 IMAP, 161 SNMP, 443 HTTPS.
1024–49151	Registered	Vendor applications, for example 3306 MySQL, 5432 PostgreSQL.
49152–65535	Dynamic / ephemeral	Chosen temporarily by the client for the duration of a connection.

Remember

A connection is identified by the **four-tuple**: source IP, source port, destination IP, destination port. Two browsers on the same machine talking to the same web server use different ephemeral source ports, which is how the OS keeps their data apart.

UDP

UDP is a minimal, connectionless datagram transport. Its header is only 8 bytes: source port, destination port, length and checksum.

Property	Consequence
Connectionless	No handshake, no state. Send immediately.
Unreliable	No acknowledgement, no retransmission, no ordering.
No flow control	The sender may outrun the receiver.
No congestion control	UDP sends regardless of network condition.
Message-oriented	It preserves message boundaries; each datagram is one message.
Small header	8 bytes versus 20 to 60 for TCP.
Checksum optional	In IPv4 the UDP checksum may be zero, i.e. disabled.

Past-paper question

Oct 2025 Q3(b)(iii). Discuss how UDP achieves low latency communication despite its unreliability.

Solution

UDP is fast for four specific reasons, and each is the *absence* of a mechanism TCP has:

1. **No handshake.** TCP spends one round trip establishing the connection before a single byte of data

moves. UDP has no such delay: the first packet carries data.

2. **No acknowledgement or retransmission.** TCD waits for ACKs and resends lost segments. Those waits and resends are latency. UDP simply moves on; if a datagram is lost, the application decides what to do.
3. **No in-order delivery guarantee.** TCP must buffer out-of-order segments until the gap is filled, which introduces *head-of-line blocking*: one lost segment stalls everything behind it even though those segments have already arrived. UDP delivers each datagram as soon as it arrives.
4. **No congestion control.** TCP reduces its rate when it detects loss. UDP keeps sending at the application's rate, so a loss does not cause a rate cut and the resulting delays.

Why that trade is right for real-time media: in a live voice or video call a frame that arrives 400 ms late is worthless. Retransmitting it wastes bandwidth *and* delays the frames behind it. It is better to drop it and carry on, which is exactly UDP's behaviour. The application (RTP over UDP, WebRTC, DNS, DHCP, TFTP, SNMP, and most online games) implements whatever reliability or ordering it actually needs.

The cost: UDP is unfair to TCP on a shared link because it does not back off, and it needs application-level mechanisms for loss, ordering and congestion, which is why the IETF later standardised QUIC (reliability over UDP, in user space) rather than letting every application reinvent it.

TCP

Features

Feature	How it is achieved
Connection-oriented	Three-way handshake before data; four-way teardown after.
Reliable	Sequence numbers, cumulative ACKs, retransmission on timeout or on three duplicate ACKs.
Ordered	The receiver buffers out-of-order segments and delivers in sequence.
Byte-stream	No message boundaries: `write()` of 100 bytes may arrive as two reads.
Full duplex	Independent data flow in both directions, each with its own window.
Flow control	Receiver advertises a window (`rwnd`).
Congestion control	Sender maintains a congestion window (`cwnd`); the effective window is the minimum of the two.
Error detection	16-bit checksum over header, data and a pseudo-header.

TCP segment header

Field	Size	Purpose
Source port	16 bits	Sender's port
Destination port	16 bits	Receiver's port
Sequence number	32 bits	Byte number of the first data byte in this segment
Acknowledgement number	32 bits	Next byte the sender expects; cumulative
Header length	4 bits	In 32-bit words; minimum 5, i.e. 20 bytes
Flags	6 bits	URG, ACK, PSH, RST, SYN, FIN
Window size	16 bits	Flow-control window advertised by the receiver
Checksum	16 bits	Over header, data and pseudo-header
Urgent pointer	16 bits	Valid only when URG is set
Options	variable	MSS, window scaling, SACK, timestamps

Connection management

Exam Focus

Three-way handshake (connection open).

1. Client sends **SYN**, `seq = x`. "I want to connect; my first byte is x."
2. Server replies **SYN+ACK**, `seq = y`, `ack = x+1`. "Accepted; my first byte is y; I have your x."
3. Client sends **ACK**, `ack = y+1`. "I have your y."

Only after step 3 does either side send data. Three messages are needed so that *each* side learns that the other's sequence number is known.

Four-way teardown (connection close).

1. Client sends **FIN**.
2. Server sends **ACK** (half-closed: the server may still send).
3. Server sends its own **FIN** when done.
4. Client sends **ACK** and waits `2MSL` before releasing the socket.

Four messages because TCP is full duplex: each direction closes independently. The `2MSL` wait ensures the final ACK is not lost and that delayed duplicate segments from the old connection cannot be mistaken for the new one.

Do Not Confuse

Why a random initial sequence number? If the sequence numbers were predictable, an off-path attacker could inject segments that the receiver would accept as genuine, and old duplicate segments from a previous connection could be accepted. Randomisation is a security control, not just a formality.

TCP versus UDP

Past-paper question

Sept 2026 CAE-1 Q6A. Compare connection-oriented and connectionless services in computer networks. Explain their characteristics, advantages, and limitations, using TCP as an example of connection-oriented service and UDP as an example of connectionless service.

Solution

Basis	TCP (connection-oriented)	UDP (connectionless)
Connection	Handshake required before data	None
Reliability	Guaranteed delivery, retransmission	Best effort, no retransmission
Ordering	In-order byte stream	No ordering guarantee
Header size	20–60 bytes	8 bytes
Speed and overhead	Slower, more state and processing	Faster, minimal state
Flow control	Yes, sliding window	No
Congestion control	Yes, slow start and AIMD	No
Data unit	Byte stream (segment)	Message (datagram)
Broadcast/multicast	Not supported	Supported
Suitable for	HTTP, FTP, SMTP, SSH, database	DNS, DHCP, VoIP, streaming, gaming, TFTP

Characteristics of a connection-oriented service: the two sides agree on a connection and maintain state; delivery is reliable and ordered; there is flow and congestion control; teardown is explicit.

Characteristics of a connectionless service: each message is independent and self-contained; no state is kept; delivery is not guaranteed; the overhead is minimal.

Advantages and limitations. TCP's advantage is correctness — applications receive a clean, ordered,

complete stream. Its limitation is latency and head-of-line blocking. UDP's advantage is latency and simplicity; its limitation is that everything TCP gives is now the application's problem.

The modern nuance worth one sentence: QUIC runs a reliable, congestion-controlled transport *over* UDP in user space, getting UDP's deployability and fast handshake with TCP's guarantees. Examiners like this.

What is the difference between connection-oriented and connectionless services

This is asked as a 3-mark standalone question. Give the table's first four rows: connection establishment, state, reliability, ordering. Add one example each.

Flow control

The sliding window in TCP

Flow control protects the **receiver**: it stops a fast sender from overrunning a slow application.

The receiver advertises a **receive window** `rwnd` in every ACK: the amount of buffer space still free. The sender keeps at most `rwnd` bytes unacknowledged. If the application stops reading and the buffer fills, `rwnd` becomes `0` and the sender stops.

Past-paper question

Oct 2025 Q3(a)(i). Discuss the major flow control mechanisms in TCP. Explain how acknowledgements and timers are used to maintain flow control.

Solution

Mechanism	Description
Advertised window	Every ACK carries the free buffer size. The sender may have at most that many bytes in flight.
Zero-window probe	When `rwnd = 0` the sender starts a persist timer; on expiry it sends a 1-byte probe so it learns when space frees up. Without this, a lost window update would deadlock the connection.
Delayed ACK	The receiver waits up to about 200 ms to piggyback the ACK on data or to ACK two segments at once.
Nagle's algorithm	The sender holds small segments until it has a full MSS or an ACK arrives, to avoid flooding the network with tiny packets.
Silly window syndrome	Stopped by *sender-side*: Nagle; and *receiver-side*: do not advertise a window smaller than an MSS until it is at least half the buffer.

How timers support this. TCP keeps four timers: the **retransmission** timer (per segment, from the measured RTT via Jacobson's algorithm), the **persist** timer (zero-window probing), the **keepalive** timer (detect a dead peer) and the **TIME_WAIT** timer (`2MSL` after closing).

ACKs and flow control together: the ACK number tells the sender what has arrived; the window field tells it how much more may be sent. Together they form a credit scheme. The sender's effective limit is

$$\text{effective window} = \min(\text{rwnd},\ \text{cwnd}) - (\text{last byte sent} - \text{last byte acked}).$$

Congestion control

How congestion occurs

Past-paper question

Oct 2025 Q3(b)(ii). Describe how congestion occurs in a network.

Solution

Congestion is a state in which the load offered to a network exceeds its capacity, so queues build and performance collapses.

The sequence is:

1. Traffic arrives at a router faster than the output link can carry it.
2. The router buffers the excess in its queue. Queueing delay rises.
3. When the queue is full, the router **drops** packets.
4. Senders time out, retransmit, and *also* slow down; but retransmissions add load.
5. Delay and loss trigger more retransmission, which adds more load. This is **congestion collapse**: throughput falls while the offered load rises.

Symptoms: rising delay and jitter, increasing loss, and throughput that falls as input grows. Note that overload is not the only cause: insufficient buffer memory, slow processors, and links whose bandwidth is mismatched in the middle of a path all produce it.

Do Not Confuse

Flow control and congestion control are not the same problem. Flow control is end-to-end and protects one receiver. Congestion control is network-wide and protects the routers. TCP implements both, using `rwnd` for the first and `cwnd` for the second, and always obeys the smaller.

Slow start, AIMD and fast recovery

Phase	Behaviour
Slow start	`cwnd` starts at 1 MSS and **doubles every RTT** (increases by one MSS per ACK). Exponential growth until the slow-start threshold.
Congestion avoidance	Above the threshold, `cwnd` grows by about one MSS per RTT: *additive increase*. The sawtooth begins here.
Timeout	`ssthresh` = half of the current `cwnd`; `cwnd` reset to 1 MSS; slow start again.
Three duplicate ACKs	Fast retransmit: the missing segment is resent immediately, without waiting for the timeout. This is **multiplicative decrease** without going back to 1.
Fast recovery	`cwnd` = `ssthresh`, then congestion avoidance. Reno behaves this way; Tahoe always restarts at 1.

Remember

The shape to draw. A curve that rises exponentially, then linearly, then drops sharply, then rises linearly again: a sawtooth. Additive increase, multiplicative decrease – AIMD. This single graph is the answer to half the congestion-control questions.

Modern variants

Variant	Idea
Tahoe	Slow start, congestion avoidance, fast retransmit. Always restarts at 1 after loss.
Reno	Adds fast recovery: after three duplicate ACKs it halves instead of restarting.
NewReno	Handles multiple losses in one window without multiple slow starts.
CUBIC	Linux default. Window growth is a cubic function of time since the last loss, so it scales to high bandwidth-delay products.
BBR	Models the bottleneck bandwidth and RTT instead of reacting to loss; performs well on lossy links such as mobile.

Quality of service

Past-paper question

Oct 2025 Q3(a)(iii). What does QoS stand for? Mention any two of its key parameters.

Solution

QoS (Quality of Service) is the set of mechanisms that let a network treat some traffic better than others, so that applications with tight requirements get predictable service.

Key parameters:

Parameter	Meaning
Bandwidth	Guaranteed minimum rate, e.g. 2 Mbps for video.
Delay (latency)	Maximum end-to-end delay, e.g. 150 ms for voice.
Jitter	Maximum variation in delay, e.g. 30 ms.
Packet loss	Maximum acceptable loss rate, e.g. 1%.
Reliability	Guaranteed delivery or availability.

Remember

Which application needs which. Voice: low delay and low jitter, modest bandwidth, tolerates some loss. Video streaming: high bandwidth, jitter-tolerant because of buffering. File transfer: no delay requirement, needs zero loss. Interactive gaming: low delay above all.

Techniques

Technique	Description
Scheduling	Priority queueing, weighted fair queueing: decide which packet goes next.
Traffic shaping	Smooth the output rate. *Leaky bucket* sends at a constant rate; *token bucket* allows bursts up to the bucket depth while enforcing an average rate.
Traffic policing	Drop or mark traffic that exceeds the profile.
Admission control	Refuse a new flow if it cannot be served.
Resource reservation	RSVP signals the required bandwidth along the path.

Model	Approach	Trade-off
IntServ	Per-flow reservation with RSVP; every router keeps state for every flow.	Precise guarantees, but does not scale to a core router with millions of flows.
DiffServ	Classify packets into a few classes (DSCP in the IP header); routers treat classes differently, keeping no per-flow state.	Scales well, but gives relative rather than absolute guarantees.

Exam Focus

Leaky versus token bucket, in one line each. A leaky bucket *forces* a constant output rate and destroys bursts. A token bucket *permits* bursts up to the bucket size and enforces only the long-run average. If the question asks which suits bursty traffic, the answer is the token bucket.

Answer skeletons

Answer Skeleton

Compare TCP and UDP (10 marks). A table of at least eight rows, then two sentences: one on when TCP is right, one on when UDP is right. Mention the four-tuple and the port ranges to show depth.

Answer Skeleton

Congestion control (10 marks). Define congestion, give the cause-and-collapse sequence, then draw the sawtooth and label slow start, congestion avoidance, fast retransmit and fast recovery. Name `cwnd` and `ssthresh` explicitly.

Answer Skeleton

Flow control (10 marks). State the problem (receiver overrun), the mechanism (advertised window), the edge cases (zero window, persist timer, silly window syndrome), and the four timers. Note that flow control is not congestion control.

Answer Skeleton

QoS (10 marks). Define QoS, list the five parameters with a number each, then contrast IntServ and DiffServ and describe one shaping mechanism.

Self-test

1. What is the difference between host-to-host and process-to-process delivery?
2. Name the three port ranges and give three well-known ports.
3. What is the four-tuple that identifies a TCP connection?
4. List the fields of the UDP header and its size.
5. Why does UDP have lower latency than TCP? Give four reasons.
6. Explain the three-way handshake with the flags and sequence numbers.
7. Why is the teardown four-way and not three-way?
8. Why must the initial sequence number be random?
9. Differentiate flow control from congestion control.
10. What is a zero-window probe and why is it needed?
11. What is silly window syndrome and how is it avoided?
12. Explain slow start and congestion avoidance.
13. What does `ssthresh` become after a timeout?
14. Distinguish fast retransmit from fast recovery.
15. State the AIMD principle and draw its sawtooth.
16. What is congestion collapse?
17. Define QoS and list five parameters.
18. Differentiate IntServ and DiffServ.
19. Compare leaky bucket and token bucket.

Answers

1. The network layer delivers to a host; the transport layer delivers to a process identified by a port.

2. Well-known 0–1023, registered 1024–49151, ephemeral 49152–65535. HTTP 80, HTTPS 443, DNS 53, SMTP 25, SSH 22.
3. Source IP, source port, destination IP, destination port.
4. Source port, destination port, length, checksum: 8 bytes total.
5. No handshake, no ACK or retransmission, no in-order buffering (so no head-of-line blocking), no congestion backoff.
6. SYN(x), SYN+ACK(y, x+1), ACK(y+1). Three messages so both sides learn the other's sequence number.
7. TCP is full duplex, so each direction is closed independently.
8. To prevent off-path injection and to avoid accepting stale duplicate segments from an earlier connection.
9. Flow control protects one receiver using `rwnd`; congestion control protects the network using `cwnd`.
10. A 1-byte probe sent when the advertised window is zero, so the sender learns when space becomes available.
11. Advertising tiny windows repeatedly; avoided by Nagle on the sender and by not advertising a window below one MSS (or half the buffer) on the receiver.
12. Slow start doubles `cwnd` each RTT until `ssthresh`; congestion avoidance then adds about one MSS per RTT.
13. Half of the `cwnd` at the time of the timeout.
14. Fast retransmit resends on three duplicate ACKs without waiting for a timeout; fast recovery continues from `ssthresh` rather than restarting at 1.
15. Additive increase, multiplicative decrease: grow linearly, halve on loss. The graph is a sawtooth.
16. A state where offered load rises but throughput falls, because retransmissions add load to an already overloaded network.
17. QoS is the set of mechanisms giving some traffic better, more predictable treatment. Parameters: bandwidth, delay, jitter, loss, reliability.
18. IntServ reserves per flow with RSVP and does not scale; DiffServ classifies into a few classes with no per-flow state and scales well.
19. A leaky bucket forces a constant rate and destroys bursts; a token bucket permits bursts up to the bucket size while enforcing the average rate.

Last-minute page

PORTS
0–1023 well known
1024–49151 registered
49152+ ephemeral
20/21 FTP, 22 SSH, 25 SMTP,
53 DNS, 80 HTTP, 443 HTTPS

UDP
8-byte header, no handshake,
no ACK, no order, no congestion
control, message oriented
DNS, DHCP, VoIP, TFTP

TCP HEADER
src/dst port, seq, ack,
header len, flags, window,
checksum, urgent, options

HANDSHAKE
SYN → SYN+ACK → ACK
Teardown 4-way, 2MSL wait

FLOW CONTROL
rwnd advertised by receiver
zero window + persist probe
Nagle, silly window syndrome

CONGESTION
cwnd, ssthresh
slow start: double per RTT
avoidance: +1 MSS per RTT
loss: halve (AIMD)
3 dup ACKs: fast retransmit

QoS
bandwidth, delay, jitter,
loss, reliability
IntServ (per flow) vs
DiffServ (classes)
leaky: constant, token: bursts

TRAPS
Flow control ≠ congestion control
Teardown is 4-way, not 3
UDP has no ordering at all

The TCP/UDP table and the congestion sawtooth are the two things to be able to produce from memory.